Planck Moment Network: Current Design and Implementation

June 4, 2026

1 Final Goal

The final goal is to train one shared neural network using the component-separated Planck patches produced by `Planck Project/compsep_planck.py`.

Each Planck patch is treated as a realization drawn from a common underlying distribution. For every patch, the component-separated Q and U maps are used as reference clean maps. Additional clean maps are synthesized from their scattering statistics, contaminated with nuisance realizations belonging to the same patch, and used to form supervised training pairs:

$$\text{input} = \text{synthetic clean map} + \text{same-patch nuisance}, \tag{1}$$

$$\text{target} = \text{synthetic clean map}. \tag{2}$$

The training pairs from all patches will eventually be pooled to train a single shared moment network. At inference time, this network should produce posterior moment maps, particularly posterior mean maps, for every patch. These final maps must have the same spatial dimensions as the original Planck patches.

2 Relevant Existing Code

The implementation was developed after examining:

- `STL_main/`, especially `Synthesis.py`, `ST_Operator.py`, and the FFT and kernel data classes;
- `Planck Project/compsep_planck.py`;
- `Moment Networks Main/`;
- `Moment Networks Pierre/`; and
- `moment_network_notes.txt`.

The current `STL_main/Synthesis.py` exposes `optimize_from_maps` and `synthesize_from_maps`. The older Moment Network scripts import an older symbol named `optimize_scattering_LBFGS`, which is not present in the current checkout. The new implementation therefore uses `optimize_from_maps`.

3 Current Files

The current implementation is located in:

```
Planck Project/Moment Network/
    make_dataset.py
    make_dataset.sbatch
```

`make_dataset.py` performs synthesis, augmentation, nuisance addition, and per-patch dataset-shard creation. It does not yet train the final shared moment network.

`make_dataset.sbatch` launches the dataset generation on a NERSC GPU node. Its parameters are written explicitly in the `export` section so that a run's configuration can be read and changed directly in the file.

4 Input Discovery

Component-separation results are discovered from:

```
/pscratch/sd/a/atsouros/STL/planck_results/version_2
```

The directory contains one NumPy result and one JSON metadata file per patch, with names such as:

```
p0_m384_fft_f64_...npy
p0_m384_fft_f64_...json
```

The patch identifier is parsed from the filename. The corresponding JSON file is used to recover the scattering-transform configuration used by the component-separation run.

Patches can be selected with `PATCH_LIST`. Accepted examples include:

```
export PATCH_LIST="0"
export PATCH_LIST="0,1,2"
export PATCH_LIST="0-15"
```

5 PBC Strategy

5.1 Original patches

The real Planck patches are not periodic. Their component-separation metadata therefore records `pbc=False`. The target scattering statistics are computed using this original non-periodic configuration.

5.2 Periodic synthesis

The synthetic running maps are generated with:

```
export SYNTHESIS_PBC="1"
export SYNTHESIS_RUNNING_SHAPE="400"
```

Thus, a typical synthesis currently uses:

Object	Shape	Boundary condition
Component-separated target	384×384	non-periodic
Running synthetic map	400×400	periodic
Final training pair	384×384	cropped from larger map

The larger periodic map permits periodic augmentation operations without placing the periodic boundary directly inside the final training image. Augmentation is performed on the larger synthetic map, after which its central region is cropped back to the original patch dimensions. Same-patch nuisance maps are added only after this final crop.

6 Augmentation

The augmentation follows the existing Moment Network implementations:

- four rotations;
- optional horizontal flip; and
- periodic translations using `torch.roll`.

Periodic rolling is appropriate for the larger periodic synthetic map. The rolled map is cropped back to the target patch size before nuisance addition. The real nuisance maps are therefore not periodically rolled.

The current test configuration uses:

```
export N_AUGMENTATIONS="8"
export SHIFT_STEP="16"
```

The number of augmentations can later be increased to 64, matching the earlier Moment Network implementations.

7 Power-Spectrum Constraint

Initially, synthesis inherited the component-separation metadata setting `st_compute_ps=True`. When the running synthesis size was changed from 384×384 to 400×400, this caused a normalization failure.

The number of default power-spectrum bins depends on map size:

$$n_{\text{bins}} = \text{int}\left(2^{\log_2(\min(\text{shape}))-4}\right). \tag{3}$$

Consequently:

$$384 \times 384 \longrightarrow n_{\text{bins}} = 24, \tag{4}$$

$$400 \times 400 \longrightarrow n_{\text{bins}} = 25. \tag{5}$$

The target operator stored power-spectrum normalization references with 24 bins, while the running operator produced 25 bins. PyTorch therefore raised:

```
RuntimeError: The size of tensor a (25) must match
the size of tensor b (24)
```

Two changes were made:

1. The running operator is explicitly given the target operator's `n_bins`, ensuring compatibility if power-spectrum constraints are enabled.
2. Power-spectrum constraints are disabled by default for the current synthesis experiment:

```
export SYNTHESIS_COMPUTE_PS="0"
```

The synthesis therefore currently matches scattering statistics without adding the power-spectrum terms to the loss. Power-spectrum constraints can be restored later by setting `SYNTHESIS_COMPUTE_PS="1"`.

8 Current Dataset Generation

For each selected patch, the script currently:

1. loads the component-separated Q and U maps;
2. loads the corresponding JSON metadata;
3. loads same-patch Q and U nuisance realizations;
4. synthesizes Q and U independently on 400×400 periodic maps;
5. augments the larger periodic synthetic maps;
6. center-crops augmented maps back to the original patch size;
7. adds a randomly selected same-patch nuisance realization; and
8. saves one compressed NumPy dataset shard for the patch.

The current small-run settings are:

```
export PATCH_LIST="0"
export PATCH_LIMIT="1"
export SYNTHESES_PER_PATCH="10"
export SYNTHESIS_BATCH_SIZE="5"
export SYNTHESIS_MAX_ITER="100"
export SYNTHESIS_PBC="1"
export SYNTHESIS_COMPUTE_PS="0"
export SYNTHESIS_RUNNING_SHAPE="400"
export N_AUGMENTATIONS="8"
```

9 Saved Dataset Keys

Each compressed `.npz` shard currently contains:

Key	Contents
`xq`	augmented/cropped clean Q targets
`yq`	contaminated Q inputs
`xu`	augmented/cropped clean U targets
`yu`	contaminated U inputs
`sq`	Q syntheses cropped to final patch size
`su`	U syntheses cropped to final patch size
`rq`	raw Q syntheses at running synthesis size
`ru`	raw U syntheses at running synthesis size
`p`	patch identifier
`stem`	source component-separation run stem
`naug`	number of augmentations
`shift`	periodic-roll shift step
`aug`	augmentation description

A separate JSON summary is also written for each patch.

10 Slurm and Cluster Environment

The scripts are expected to be located on the cluster at:

```
/pscratch/sd/a/atsouros/STL/Moment Network
```

The repository root used for importing `STL_main` is:

```
/pscratch/sd/a/atsouros/STL
```

The sbatch file uses the NERSC account and GPU configuration already used by the Planck component-separation scripts:

```
#SBATCH -A mp107d_g
#SBATCH --constraint=gpu&hbm80g
#SBATCH --qos=debug
#SBATCH --gpus-per-task=1
```

The log filenames are fixed:

```
#SBATCH --output=mn_dataset.log
#SBATCH --error=mn_dataset.err
```

The default system Python versions on the login node are too old:

```
python --version # Python 2.7.18
python3 --version # Python 3.6.15
```

The sbatch file therefore loads and activates the existing `cmbenv` environment before invoking `python3`. Unbound-variable checking is temporarily disabled during environment activation because the conda activation scripts reference variables that may initially be unset.

11 Work Still Required

The current implementation only creates per-patch dataset shards. The following stages are still required:

1. create a pooled or lazy dataset loader over all patch shards;
2. define train and validation splits, including held-out-patch validation;
3. train one shared Q network and one shared U network, or evaluate a joint two-channel Q/U network;
4. run inference on every original observed patch;
5. save posterior mean and posterior standard-deviation maps as full-size `.npy` files; and
6. evaluate full-map and boundary-sensitive metrics.

12 Important Validation Considerations

Randomly splitting augmented samples can place samples originating from the same patch in both training and validation sets. This is useful for monitoring optimization but does not measure generalization to unseen patches. A held-out-patch validation split should therefore be used in addition to any random sample split.

The effect of periodic synthesis should also be evaluated. Although the final training maps are cropped from larger periodic maps, comparisons should be made between full-map metrics and interior-region metrics to detect remaining boundary effects.